

Course Title : Data Structures and Algorithms

Assignment Number : PGDCA_NEW(2)/208/Assignment/2023-24

Maximum Marks : 100

Weightage : 30% www.ignou.site.com

Last Date of Submission : 31st October, 2023 (for July 2023 session)

: 30th April, 2024 (for January 2024 session)



Note: - This assignment only for students, not for sell or re-upload any media or website. All right reserve to "IGNOU Study Helper". It is illegal to share or reupload it. If anything like this is found, then appropriate action will be taken and apply copyright ACT to you. You will be responsible for illegal work. So don't share and upload on any media.

Question 1: What are B-trees? Explain with example.

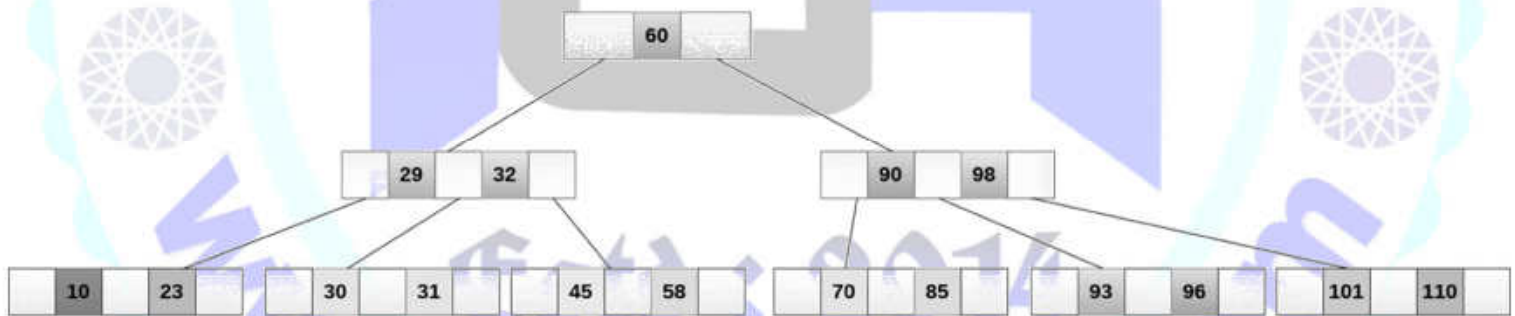
Ans. B Tree: B Tree is a specialized m-way tree that can be widely used for disk access. A B-Tree of order m can have at most m-1 keys and m children. One of the main reason of using B tree is its capability to store large number of keys in a single node and large key values by keeping the height of the tree relatively small.

A B tree of order m contains all the properties of an M way tree. In addition, it contains the following properties.

1. Every node in a B-Tree contains at most m children.
2. Every node in a B-Tree except the root node and the leaf node contain at least $m/2$ children.
3. The root nodes must have at least 2 nodes.
4. All leaf nodes must be at the same level.

It is not necessary that, all the nodes contain the same number of children but, each node must have $m/2$ number of nodes.

A B tree of order 4 is shown in the following image.



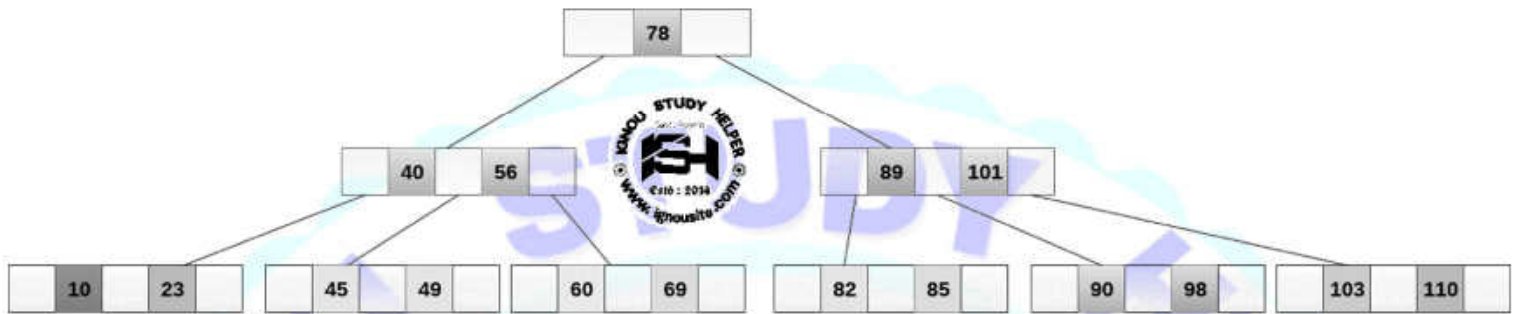
While performing some operations on B Tree, any property of B Tree may violate such as number of minimum children a node can have. To maintain the properties of B Tree, the tree may split or join.

Operations

Searching :

Searching in B Trees is similar to that in Binary search tree. For example, if we search for an item 49 in the following B Tree. The process will something like following :

1. Compare item 49 with root node 78. since $49 < 78$ hence, move to its left sub-tree.
2. Since, $40 < 49 < 56$, traverse right sub-tree of 40.
3. $49 > 45$, move to right. Compare 49.
4. match found, return.



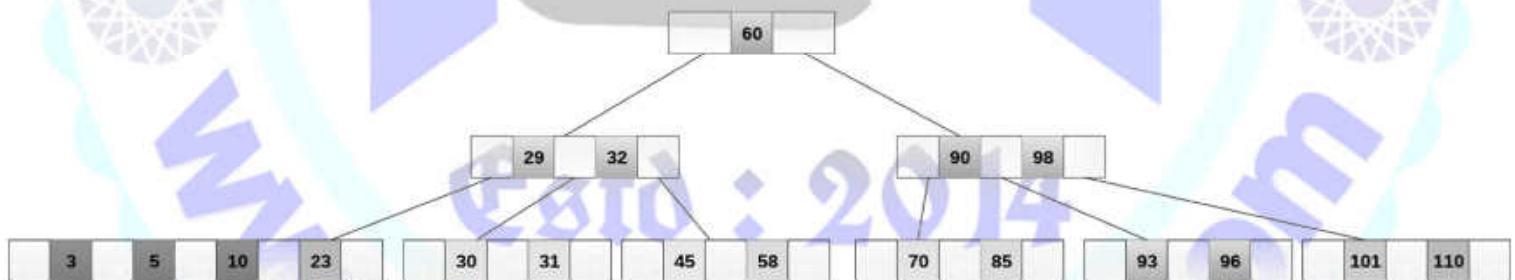
Inserting

Insertions are done at the leaf node level. The following algorithm needs to be followed in order to insert an item into B Tree.

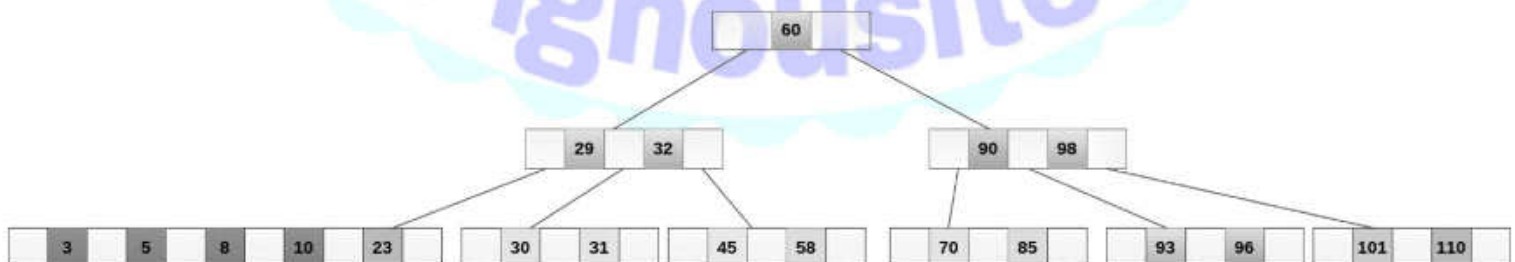
1. Traverse the B Tree in order to find the appropriate leaf node at which the node can be inserted.
2. If the leaf node contain less than $m-1$ keys then insert the element in the increasing order.
3. Else, if the leaf node contains $m-1$ keys, then follow the following steps.
 - Insert the new element in the increasing order of elements.
 - Split the node into the two nodes at the median.
 - Push the median element upto its parent node.
 - If the parent node also contain $m-1$ number of keys, then split it too by following the same steps.

Example:

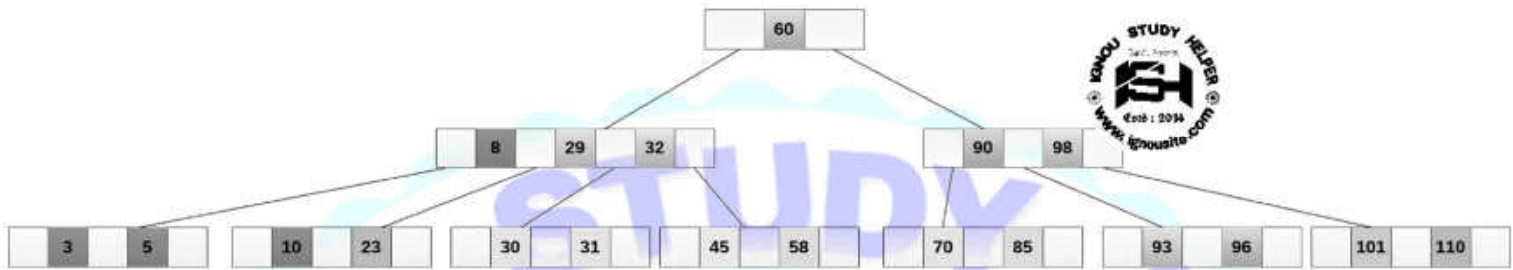
Insert the node 8 into the B Tree of order 5 shown in the following image.



8 will be inserted to the right of 5, therefore insert 8.



The node, now contain 5 keys which is greater than $(5 - 1 = 4)$ keys. Therefore split the node from the median i.e. 8 and push it up to its parent node shown as follows.



Deletion

Deletion is also performed at the leaf nodes. The node which is to be deleted can either be a leaf node or an internal node.

Following algorithm needs to be followed in order to delete a node from a B tree.

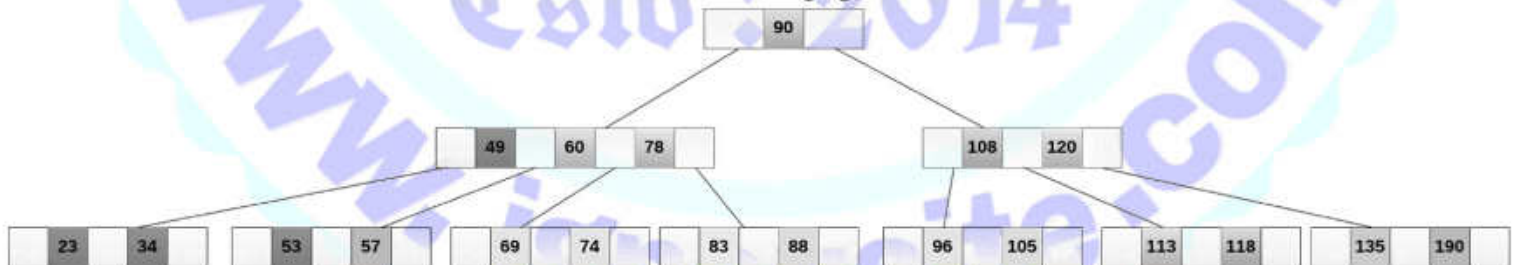
1. Locate the leaf node.
2. If there are more than $m/2$ keys in the leaf node then delete the desired key from the node.
3. If the leaf node doesn't contain $m/2$ keys then complete the keys by taking the element from right or left sibling.
 - If the left sibling contains more than $m/2$ elements then push its largest element up to its parent and move the intervening element down to the node where the key is deleted.
 - If the right sibling contains more than $m/2$ elements then push its smallest element up to the parent and move the intervening element down to the node where the key is deleted.
4. If neither of the sibling contain more than $m/2$ elements then create a new leaf node by joining two leaf nodes and the intervening element of the parent node.
5. If parent is left with less than $m/2$ nodes then, apply the above process on the parent too.

If the the node which is to be deleted is an internal node, then replace the node with its in-order successor or predecessor.

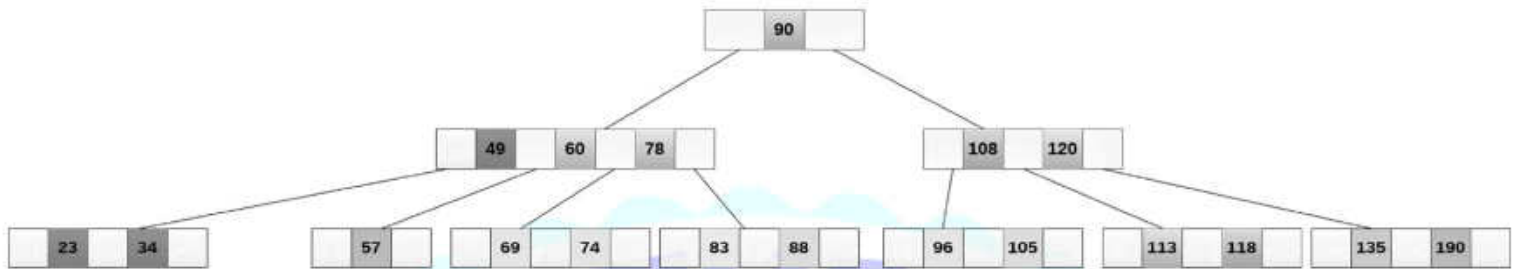
Since, successor or predecessor will always be on the leaf node hence, the process will be similar as the node is being deleted from the leaf node.

Example 1

Delete the node 53 from the B Tree of order 5 shown in the following figure.

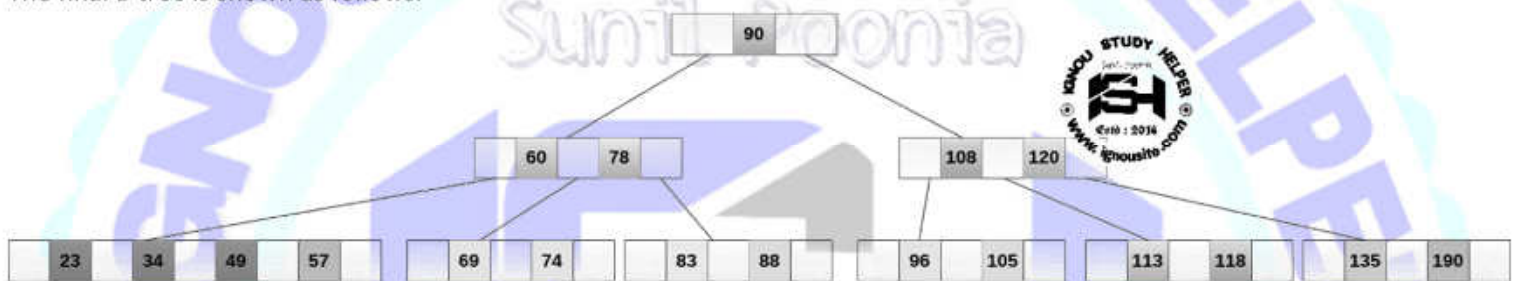


53 is present in the right child of element 49. Delete it.



Now, 57 is the only element which is left in the node, the minimum number of elements that must be present in a B tree of order 5, is 2. it is less than that, the elements in its left and right sub-tree are also not sufficient therefore, merge it with the left sibling and intervening element of parent i.e. 49.

The final B tree is shown as follows.



Question 2: Explain the process of converting a Tree into a Binary Tree with an example.

Ans. Process of converting a Tree into a Binary Tree: A general tree is an unordered hierarchical data structure with unlimited children nodes for each parent. A binary tree only has a maximum of two children nodes for each parent, commonly called left node and right node.

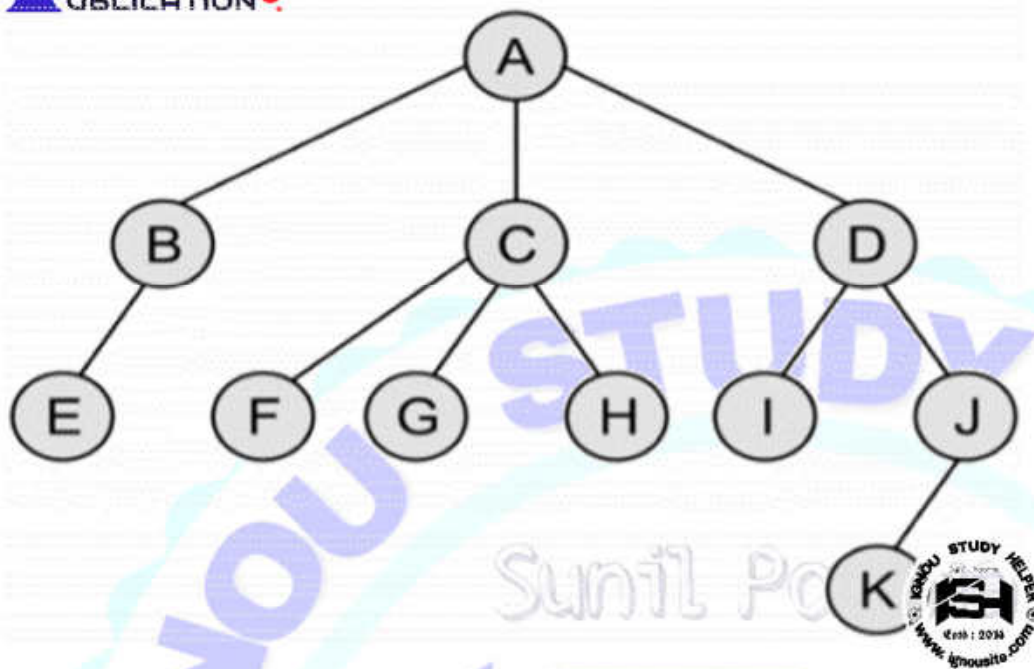
Converting from tree to binary tree would simply require that you start with the root node and start copying each node into the binary tree. The only rules would be that each node cannot have more than two children, including the root node.

Following are the rules to convert a Tree to a Binary Tree:

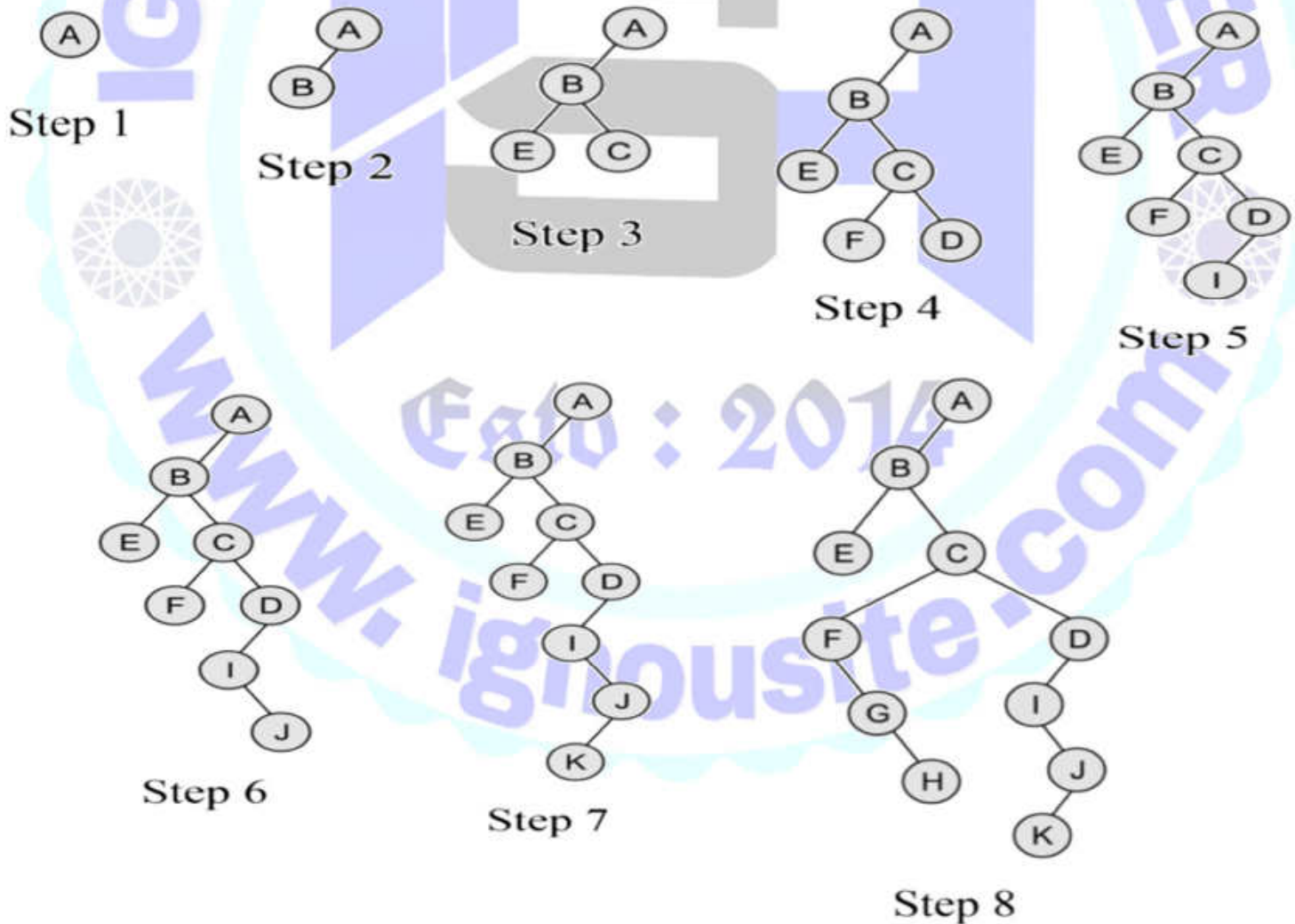
- The root of the Binary Tree is the Root of the Generic Tree.
- The left child of a node in the Generic Tree is the Left child of that node in the Binary Tree.
- The right sibling of any node in the Generic Tree is the Right child of that node in the Binary Tree.

Examples:

Convert the following Generic Tree to Binary Tree:



Below is the Binary Tree of the above Generic Tree:



Below are the steps for the conversion of Generic Tree to Binary Tree:

1. As per the rules mentioned above, the root node of general tree A is the root node of the binary tree.
2. Now the leftmost child node of the root node in the general tree is B and it is the leftmost child node of the binary tree.
3. Now as B has E as its leftmost child node, so it is its leftmost child node in the binary tree whereas it has C as its rightmost sibling node so it is its right child node in the binary tree.
4. Now C has F as its leftmost child node and D as its rightmost sibling node, so they are its left and right child node in the binary tree respectively.
5. Now D has I as its leftmost child node which is its left child node in the binary tree but doesn't have any rightmost sibling node, so doesn't have any right child in the binary tree.
6. Now for I, J is its rightmost sibling node and so it is its right child node in the binary tree.
7. Similarly, for J, K is its leftmost child node and thus it is its left child node in the binary tree.
8. Now for C, F is its leftmost child node, which has G as its rightmost sibling node, which has H as its just right sibling node and thus they form their left, right, and right child node respectively.

Program Example:

```
#include <iostream>
#include <vector>

class TreeNode {
public:
    int val;
    TreeNode* left;
    TreeNode* right;
    std::vector<TreeNode*> children;
    TreeNode(int val)
    {
        this->val = val;
        this->left = this->right = nullptr;
    }
};

TreeNode* convert(TreeNode* root)
{
    if (!root) {
        return nullptr;
    }

    if (root->children.size() == 0) {
        return root;
    }

    if (root->children.size() == 1) {
        root->left = convert(root->children[0]);
        return root;
    }
}
```



```

root->left = convert(root->children[0]);
root->right = convert(root->children[1]);

for (int i = 2; i < root->children.size(); i++) {
    TreeNode* rightTreeRoot = root->right;
    while (rightTreeRoot->left != nullptr) {
        rightTreeRoot = rightTreeRoot->left;
    }
    rightTreeRoot->left = convert(root->children[i]);
}

return root;
}

void printTree(TreeNode* root)
{
    if (!root) {
        return;
    }
    std::cout << root->val << " ";
    printTree(root->left);
    printTree(root->right);
}

int main()
{
    TreeNode* root = new TreeNode(1);
    root->children.push_back(new TreeNode(2));
    root->children.push_back(new TreeNode(3));
    root->children.push_back(new TreeNode(4));
    root->children.push_back(new TreeNode(5));

    root->children[0]->children.push_back(new TreeNode(6));
    root->children[0]->children.push_back(new TreeNode(7));
    root->children[3]->children.push_back(new TreeNode(8));
    root->children[3]->children.push_back(new TreeNode(9));

    TreeNode* binaryTreeRoot = convert(root);

    printTree(binaryTreeRoot);
}

```

Output:

1 2 3 4 5 6 7 8 9

Question 3: What is Heap Sort? What is Merge Sort? Write the factors on the basis of which Heap Sort or Merge Sort is selected.

Ans. Heap Sort: Heap sort is a comparison-based sorting technique based on Binary Heap data structure. It is similar to the selection sort where we first find the minimum element and place the minimum element at the beginning. Repeat the same process for the remaining elements.

Heap Sort Algorithm

To solve the problem follow the below idea:

First convert the array into heap data structure using heapify, then one by one delete the root node of the Max-heap and replace it with the last node in the heap and then heapify the root of the heap. Repeat this process until size of heap is greater than 1.

- Build a heap from the given input array.
- Repeat the following steps until the heap contains only one element:
 - Swap the root element of the heap (which is the largest element) with the last element of the heap.
 - Remove the last element of the heap (which is now in the correct position).
 - Heapify the remaining elements of the heap.
- The sorted array is obtained by reversing the order of the elements in the input array.

Merge Sort: Merge sort is defined as a sorting algorithm that works by dividing an array into smaller subarrays, sorting each subarray, and then merging the sorted subarrays back together to form the final sorted array.

Merge Sort Algorithm

In the following algorithm, arr is the given array, beg is the starting element, and end is the last element of the array.

MERGE_SORT(arr, beg, end)

if beg < end

set mid = (beg + end)/2

MERGE_SORT(arr, beg, mid)

MERGE_SORT(arr, mid + 1, end)

MERGE (arr, beg, mid, end)

end of if

END MERGE_SORT

Factors on the basis of which Heap Sort or Merge Sort: Heap Sort and Merge Sort are both efficient sorting algorithms, but they have different characteristics and are chosen based on various factors depending on the specific requirements of a problem. Here are the factors on the basis of which Heap Sort or Merge Sort might be selected:

1. Time Complexity:

Heap Sort has a time complexity of $O(n \cdot \log(n))$ in the worst, average, and best cases, making it a good choice for sorting large datasets. It is particularly useful when a stable sort is not required.

Merge Sort also has a time complexity of $O(n \cdot \log(n))$ in all cases. It is stable and works well with larger datasets.

2. Space Complexity:

Heap Sort typically has a space complexity of $O(1)$ as it sorts the data in place, making it a space-efficient algorithm.

Merge Sort, on the other hand, requires additional memory space for the merging step, resulting in a space complexity of $O(n)$.

3. Stability:

If maintaining the relative order of equal elements (stability) is important, Merge Sort is preferred as it is a stable sorting algorithm. Heap Sort is not inherently stable.

4. Implementation Complexity:

Implementing Heap Sort is usually simpler and requires less code than Merge Sort, making it an attractive option when simplicity is a priority.

5. Data Structure:

Heap Sort relies on the concept of a binary heap, which is an efficient data structure for this algorithm. If the data is already in a heap-like structure, Heap Sort can be a natural choice.

Merge Sort doesn't have specific data structure requirements and can work well with various data structures.

6. Adaptive Behavior:

Heap Sort does not adapt well to partially sorted datasets. It maintains the same time complexity regardless of the initial order of the data.

Merge Sort performs consistently well even with partially sorted data, as its time complexity remains $O(n \log n)$.

7. In-Place Sorting:

If sorting needs to be done in-place without using additional memory, Heap Sort is preferred due to its $O(1)$ space complexity. Merge Sort requires additional memory for the merging step, so it is not an in-place sorting algorithm.

8. Stability:

If the relative order of equal elements is important, Merge Sort is preferred as it is a stable sorting algorithm. Heap Sort is not inherently stable.

Question 4: What is a Doubly Linked List? How does it differ from Circularly Doubly Linked List?

Ans. Doubly linked list: Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer), pointer to the previous node (previous pointer). A sample node in a doubly linked list is shown in the figure.



Node

A doubly linked list containing three nodes having numbers from 1 to 3 in their data part, is shown in the following image.



Doubly Linked List

Differ from Circularly Doubly Linked List: Doubly Linked List and Circular Doubly Linked List are two variations of linked lists with some differences:

1. Basic Structure:

Doubly Linked List (DLL): In a doubly linked list, each node contains two pointers/references, one pointing to the next node (usually called next or forward pointer) and another pointing to the previous node (usually called prev or backward pointer).

Circular Doubly Linked List (CDLL): A circular doubly linked list is similar to a doubly linked list, but it forms a closed loop. In a CDLL, the next pointer of the last node points back to the first node, creating a circular structure.

2. Termination Condition:

Doubly Linked List (DLL): In a regular doubly linked list, the prev pointer of the first node and the next pointer of the last node are typically set to NULL to indicate the beginning and end of the list.

Circular Doubly Linked List (CDLL): In a circular doubly linked list, there is no NULL pointer at the beginning or end. The list is considered circular because the last node's next pointer points back to the first node, and the first node's prev pointer points to the last node.

3. Traversal:

Doubly Linked List (DLL): To traverse a doubly linked list, you can move both forwards and backward through the list since each node has both next and prev pointers.

Circular Doubly Linked List (CDLL): Similarly, in a circular doubly linked list, you can traverse the list in both directions (forward and backward) by following the next and prev pointers.

4. Use Cases:

Doubly Linked List (DLL): Doubly linked lists are commonly used when you need to perform operations that require both forward and backward traversal, such as implementing undo/redo functionality or certain types of data structures like deques.

Circular Doubly Linked List (CDLL): Circular doubly linked lists are useful when you want to create a looped structure, like modeling a circular buffer or a round-robin scheduling algorithm.

5. Termination Check:

Doubly Linked List (DLL): In a doubly linked list, it's easy to check for the end of the list by examining the next pointer. If it's NULL, you've reached the end.

Circular Doubly Linked List (CDLL): In a circular doubly linked list, you need to be cautious when traversing since there's no natural end (no NULL pointer). You need to establish some criteria to stop the traversal.

Doubly Linked List (DLL): Insertion and deletion in a doubly linked list involve updating both next and prev pointers, which allows for easy removal of elements from anywhere in the list.

Circular Doubly Linked List (CDLL): Insertion and deletion in a circular doubly linked list are also similar to a doubly linked list but require extra care when handling the first and last nodes to maintain the circular structure.

